

Design Pattern

Observer

Adam HAMOUCH

May 2026

1. Intention

Permet a des objets (**subscribers**) de s'abonner a un autre objet (**publisher**) et d'être notifiés automatiquement quand un événement survient.

Aussi appelle : **Publish/Subscribe**, **Event system**, **Signal/Slot** (Qt), **Delegate** (Unreal).

2. Structure

- **Publisher** : detient la liste des subscribers, expose `subscribe()/unsubscribe()`, itere pour notifier.
- **Subscriber** : s'abonne et reagit a la notification.
- Les subscribers **ne se connaissent pas** — le publisher est l'unique point de contact.

3. Implementation 1 — Interface IObserver

Tous les subscribers heritent d'une interface commune. Liste homogene en type, heterogene en comportement.

```
struct Event { std::string type; float value; };

class IObserver {
public:
    virtual void onEvent(const Event& e) = 0;
    virtual ~IObserver() = default;
};

class AudioSystem : public IObserver {
public:
    void onEvent(const Event& e) override {
        if (e.type == "EnemyDied") playDeathSound();
    }
};

class UISystem : public IObserver {
public:
    void onEvent(const Event& e) override {
        if (e.type == "PlayerHit") updateHealthBar(e.value);
    }
};

class EventBus {
public:
    void subscribe(IObserver* o) { observers.push_back(o); }
    void unsubscribe(IObserver* o) {
        observers.erase(std::remove(observers.begin(),
                                    observers.end(), o));
    }
    void notify(const Event& e) {
        for (auto* o : observers) o->onEvent(e);
    }
private:
    std::vector<IObserver*> observers;
};
```

```

EventBus bus;
AudioSystem audio; UISystem ui;
bus.subscribe(&audio);
bus.subscribe(&ui);
bus.notify({"EnemyDied", 0.f});

```

4. Implementation 2 — std::function (moderne)

Plus flexible : n'importe quel callable s'abonne sans heritage.

```

class EventBus {
public:
    void subscribe(std::function<void(const Event*)> cb) {
        callbacks.push_back(cb);
    }
    void notify(const Event& e) {
        for (auto& cb : callbacks) cb(e);
    }
private:
    std::vector<std::function<void(const Event*)>> callbacks;
};

bus.subscribe([](const Event& e) { playDeathSound(); });
bus.subscribe([&ui](const Event& e) { ui.updateHealthBar(e.value); });

```

Unreal abstrait tout cela via les Delegates :

```

DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOnDied, AEnemy*, Enemy);
class AEnemy : public AActor {
public:
    UPROPERTY(BlueprintAssignable)
    FOnDied OnDied;
    void Die() { OnDied.Broadcast(this); } // Unreal itere pour toi
};
Enemy->OnDied.AddDynamic(this, &AMyClass::HandleDeath);

```

5. Ownership — weak_ptr pour eviter les dangling

Si un subscriber est detruit avant de se desabonner, le publisher itere un pointeur invalide. Solution : stocker des **weak_ptr** et verifier avant chaque notification.

```

class EventBus {
    std::vector<std::weak_ptr<IObserver>> observers;
public:
    void subscribe(std::shared_ptr<IObserver> o) { observers.push_back(o); }
    void notify(const Event& e) {
        for (auto it = observers.begin(); it != observers.end(); ) {
            if (auto sp = it->lock()) { sp->onEvent(e); ++it; }
            else it = observers.erase(it); // subscriber mort, on nettoie
        }
    }
};

```

6. Cas d'usage / Quand utiliser

Utiliser :

- Mort ennemi → notifie audio, UI, score, achievements simultanement.
- Changement d'etat (pause, game over) → notifie tous les systemes.
- Collision → notifie physique, son, effets visuels.

Eviter :

- Hot path critique (boucle de rendu) — overhead d'iteration et de vtable.
- Si les dependances entre systemes sont simples et fixes — injection directe suffit.